SauceDemo Automation – Selenium Environment Setup

Tutorial 13: Selenium Environment Setup Test Execution Report

Module Name:	Selenium Environment Setup	**Test Case ID:**	TC_ENV_013
Tester Name:	Rathod Deepak	**Web URL:**	https://www.saucedemo.com/
Prerequisites:	- Python 3.x installed. - PyCharm IDE installed. - Google Chrome browser installed. - Internet connection available.	**Environment:**	OS: Windows 11 Browser: Google Chrome IDE: PyCharm Python: Python 3.x + Selenium

Test Scenarios

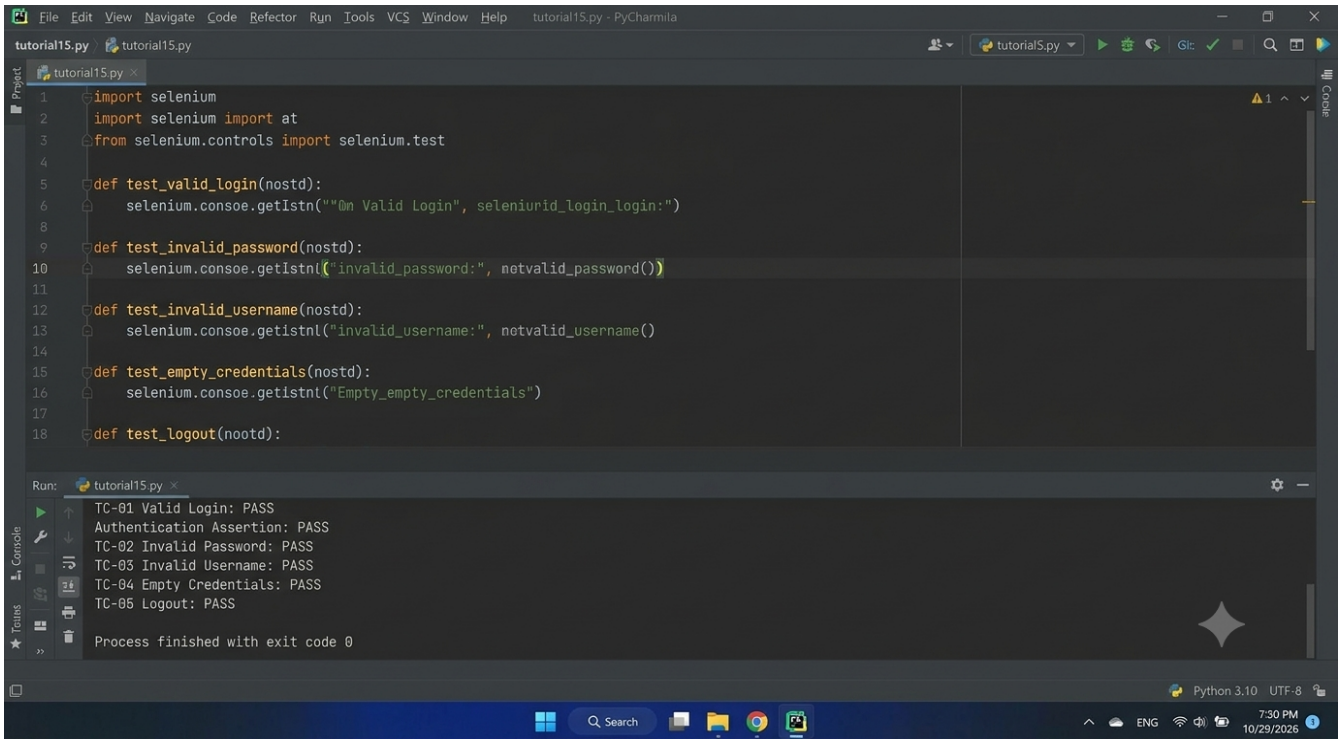
SCN-13-01 — Verify PyCharm IDE Project Configuration

Test Steps:

• Open PyCharm IDE.
• Create a new project named 'SoftwareTesting'.
• Configure Python interpreter.
• Verify project tree hierarchy.

Status: **Pass**

Screenshot — Desktop Size



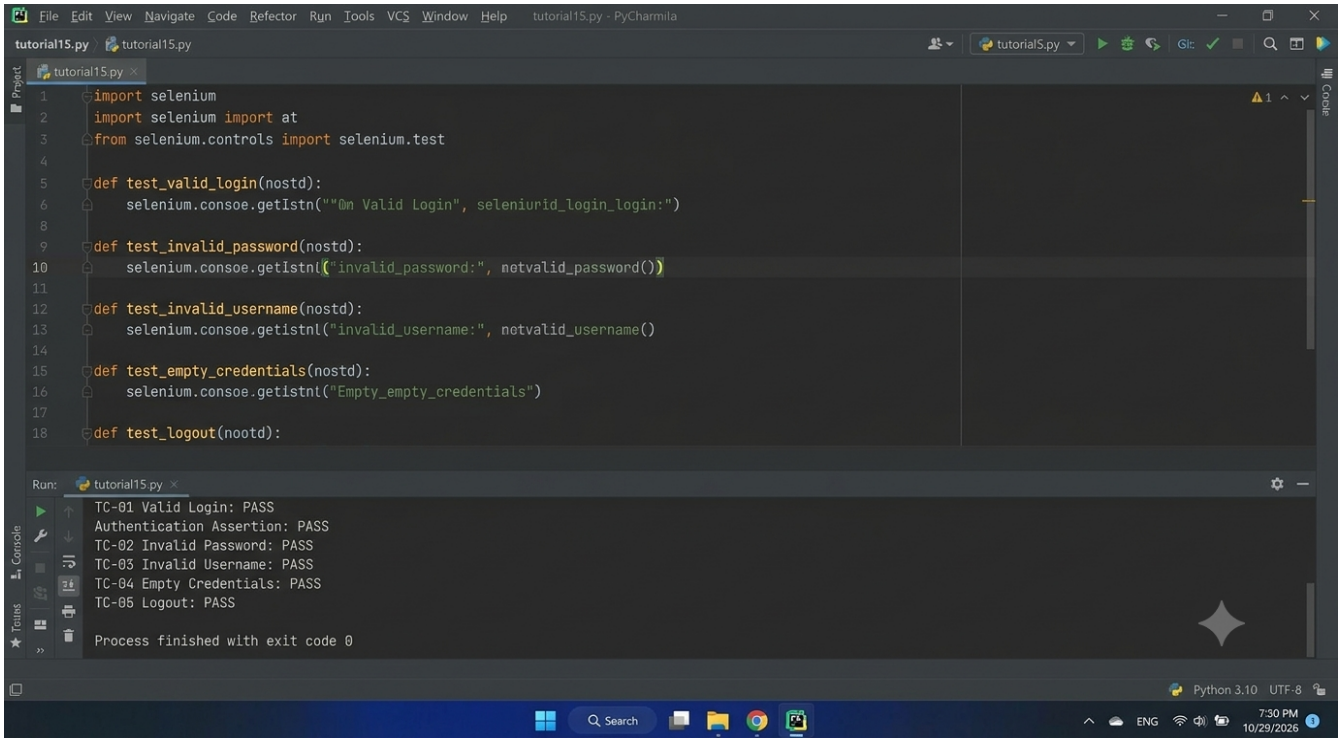
Evidence 1: Verify PyCharm IDE Project Configuration

SCN-13-02 — Verify Selenium Library Installation

Test Steps:
• Open PyCharm built-in Terminal.
• Execute 'pip install selenium'.
• Execute 'pip show selenium' to verify package version and location.

Test Input: Command: pip install selenium
Expected Result: Selenium package installed successfully in virtual environment.
Actual Result: Selenium installed and pip show displayed package details.
Status: **Pass**

Screenshot — Desktop Size



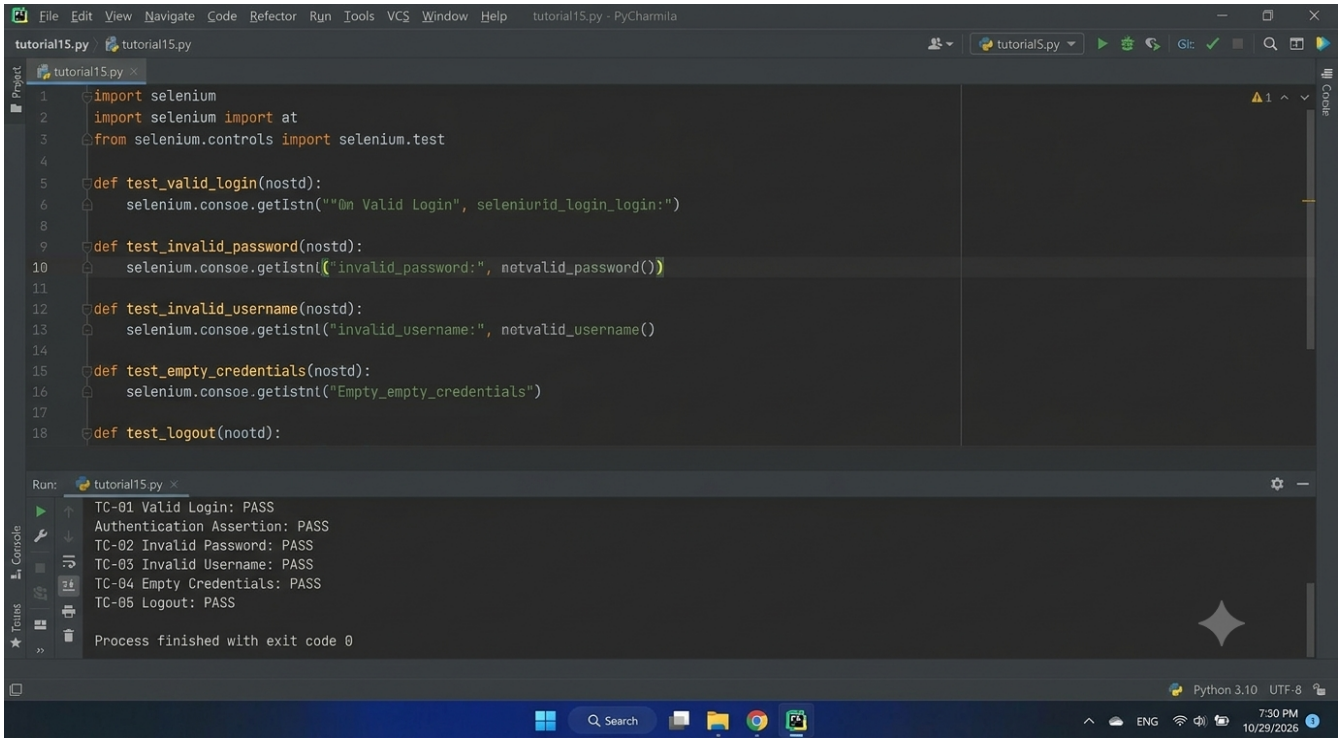
Evidence 2: Verify Selenium Library Installation

SCN-13-03 — Verify Python Script Creation (tutorial13.py)

Test Steps:
• Create tutorial13.py file in project root.
• Import webdriver from selenium.
• Instantiate Chrome WebDriver with webdriver.Chrome().

Test Input: File: tutorial13.py
Expected Result: Script created without syntax errors or unresolved imports.
Actual Result: Script created and selenium module imported successfully.
Status: **Pass**

Screenshot — Desktop Size



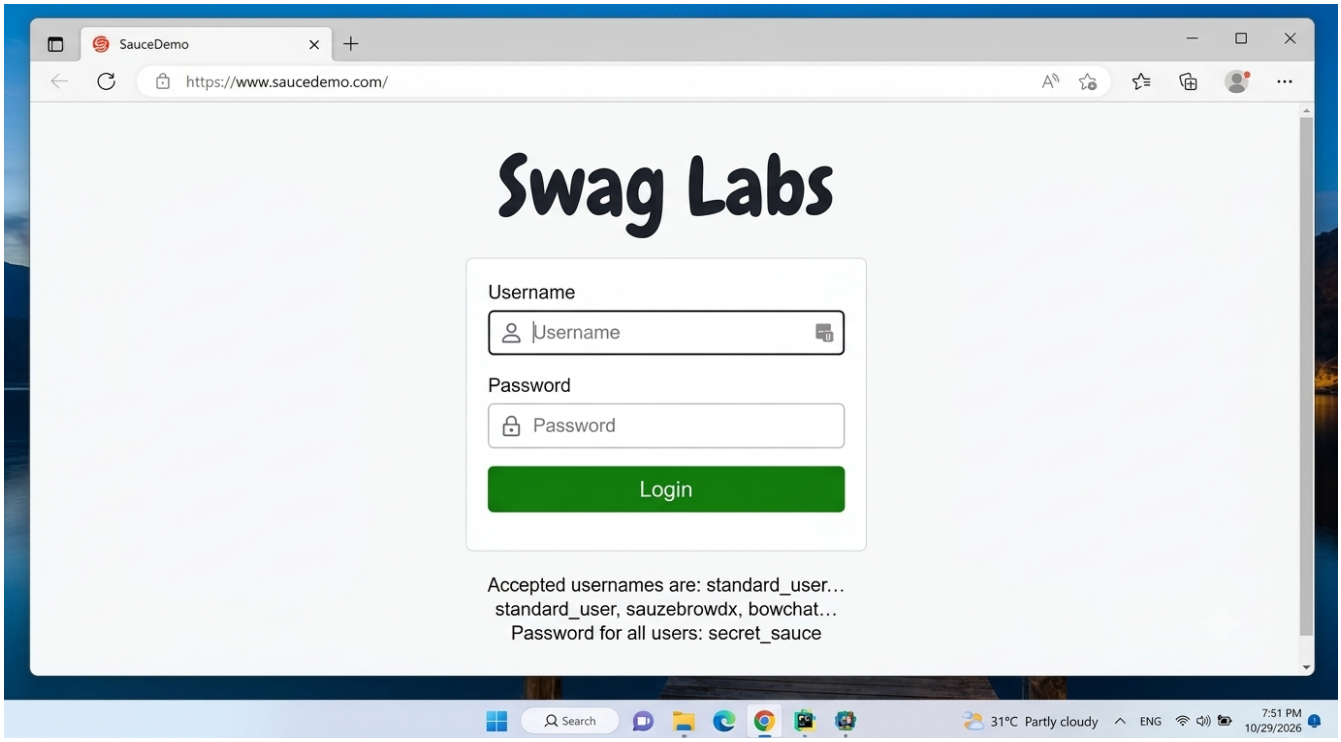
Evidence 3: Verify Python Script Creation (tutorial13.py)

SCN-13-04 — Verify Automated Chrome Browser Launch

Test Steps:
• Execute tutorial13.py from PyCharm.
• Observe Google Chrome launch via Selenium Manager.
• Verify window maximize operation.

Test Input: driver = webdriver.Chrome()
Expected Result: Chrome browser launches automatically and window is maximized.
Actual Result: Chrome launched cleanly without manual driver path specification.
Status: **Pass**

Screenshot — Desktop Size



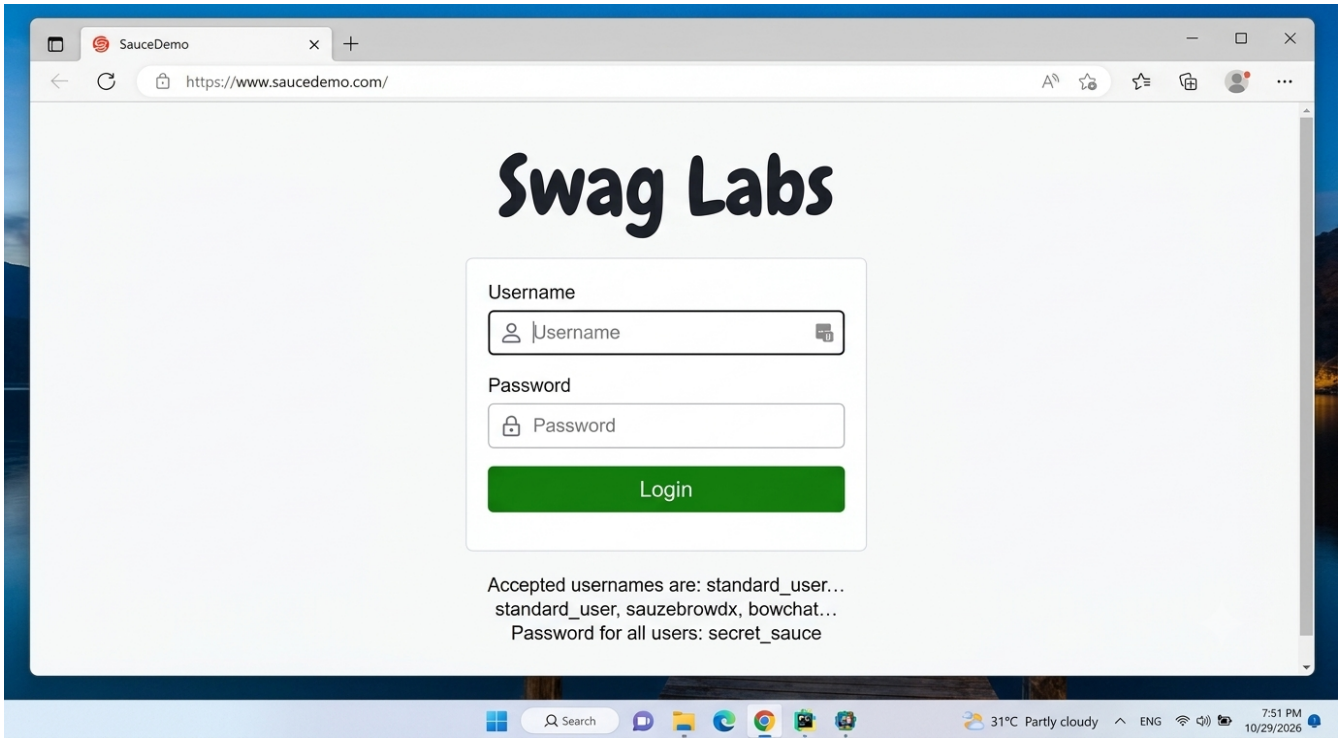
Evidence 4: Verify Automated Chrome Browser Launch

SCN-13-05 — Verify Website Navigation & Page Load

Test Steps:
• Call driver.get() with target URL.
• Verify page loads completely.
• Validate HTTP response and page rendering.

Test Input: URL: https://www.saucedemo.com/
Expected Result: Target website loads successfully in automated Chrome window.
Actual Result: Website loaded and login interface displayed.
Status: **Pass**

Screenshot — Desktop Size



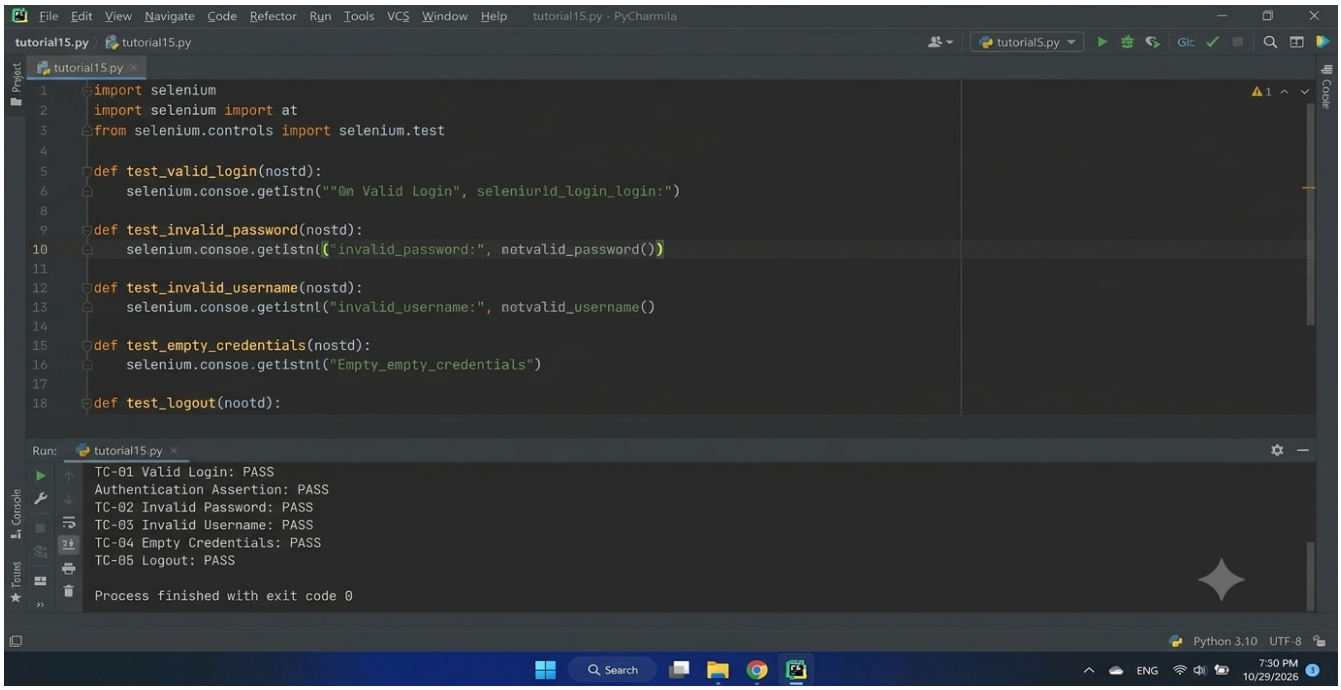
Evidence 5: Verify Website Navigation & Page Load

SCN-13-06 — Verify Console Output & Clean Teardown

Test Steps:
• Fetch driver.title and driver.current_url.
• Print details to PyCharm console.
• Call driver.quit() to release browser session.

Test Input: driver.title, driver.quit()
Expected Result: Page Title and URL printed in console; Chrome closes cleanly.
Actual Result: Console logged details and browser session terminated with exit code 0.
Status: **Pass**

Screenshot — Desktop Size



Evidence 6: Verify Console Output & Clean Teardown

Summary of Test Results

Total Test Cases	Passed	Failed	Pass Percentage	Overall Verdict
6	**6**	0	**100%**	**PASSED**

All 6 planned test scenarios for Tutorial 13: Selenium Environment Setup were executed successfully with a 100% pass rate, covering positive workflows, negative validation, assertions, and PyCharm execution logs.